

ng 1

2. Prevent approval by author is TRUE

setting 2

Note: The Approvals required project setting is also a workaround to make the CI variable \$CIMERGEREQUESTAPPROVED work correctly, see issue for more details.

SECTION: enterprise-data/how-we-work/new-data-source.md

This page describes in detail the process of adding new data sources to the data warehouse. The Data Team embraces any initiative of adding new data sources towards the data warehouse. We envision that data could lead towards value, but adding new data towards the data warehouse does not go with no costs.

Both the development (assigning resources from the Data Team, from other teams involved to support and also you as requestor) and keeping the data pipeline up and running (storage, compute resources, incident management, monitoring etc..) cost time and or money.

Please, before request adding a new data source, take the following into account:

- Is there a valid business case?
 - Sometimes the business case is to comply with regulatory requirements.
 - Sometimes the business case is straightforward because value potential is clear above the investment costs.
 - Most of the time it's hard to quantify, because either the value is unsure or the costs are unsure. Feel free to have an open and honest discussion with the Data Team. We have the experience and could help with the justification and this does not necessarily need to be a scientific calculation.
- The work is not done when data lands in the data warehouse. The data will 'just' land in the raw data layer, and this is not accessible by default for Sisense. Data need to be loaded downstream into the Enterprise Dimensional Model (EDM) via dbt. Follow up needs to take place, and will come on top of the process described on this page.
 - Downstream modelling could be handled by the business team, because we embrace contribution on our data platform. Please note that extensive (dbt-, SQL- and data modelling knowledge is needed)
 - Downstream modelling could be handled by the Data Team, ideally by a Data Fusion Team. Planning needs to take place and priorities are set in line with company priorities. This falls not within the scope of adding a new data source to the data warehouse, thus this needs to be arranged subsequently.
- If data is extracted in (complex) JSON format, a Data Engineer can support or can flatten the data to a tabular format and load this towards the PREP database.
- There are 3 ways to follow up on the downstream data development.
- Adding a new data source to the data warehouse is not an 1 off exercise. As soon as the data is extracted to the data warehouse, on a regular cadence (once week, once a day, multiple times per day, etc..) data will be refreshed. This means something can happen or can go wrong after the implementation. We will need a DRI from the source side (business and technical) to support

this process when needed.

- Data could be used, outside of the EDM. I.e. by a Function Analyst, in the raw data layer.

Raise an AR to get access to the raw data.

- When data ends up in the EDM, work is to be performed in Sisense by creating a dashboard. Also for doing this, some technical knowledge is required.

- The data platform needs to get access to the source system. Although this sounds obvious, this is not always straightforward.

New data source

For adding a new data source, the workflow looks as follow (note, this is the existing workflow we use for all our work and made explicit here in the context of adding a new data source).

Process for adding a new data source:

Stage (Label)	Summary for new data source
-----	-----
workflow::1 - triage	All initial information will be provided by the requester and assessed by the Data Team
workflow::2 - validation	A solution design and complete work breakdown for upcoming development will be created
workflow::3 - scheduling	Once a quarter the Data Team OKRs are defined. Based on workload and priorities, new data sources could be incorporated in the next quarter OKRs
workflow::4 - scheduled	Adding a new data source is on the list of next quarter OKRs. An epic will be created with all the work breakdown activities attached
workflow::5 - development	Development is in progress
workflow::6 - review	Development is under review
workflow::X - blocked	Development is blocked

workflow::1 - triage

Every new data source requests starts with an issue. This applies for data source that are already extracted, but needs to be extended and for completely new data sources. Please use the New Data Source template. Assign the new issue to the Manager, Data Platform so that the Data Team can triage the issue in a timely manner. All initial information will be provided by the requestor and assessed by the Data Team. Cross check will be performed if the data doesn't already exist in the data warehouse.

workflow::2 - validation

All details regarding the new data source will be flashed out, with the goal of creating a full work breakdown.

Data scope

Based on the requirements, the data points that need to be extracted (i.e. which tables) will be determined. A total data scope must be given.

Data minimisation

In the Data Platform we adopt the principle of data minimisation when extracting data, which means we only extract data that is needed to fulfill the scope/need. We do this for the following reasons:

- Less data is using less resources, reducing pressure on source systems, pipelines and our data platform.
- Less data means less maintenance, if a data source changes and we don't extract that particular data(set), we likely don't need to do anything.

This is all in the spirit of GitLabs value of efficiency.

Data minimisation is applied on the following levels:

- Data sources. We only extract data sources that are needed.
- Tables. We only extract the tables within an extraction that are needed.
- Columns. We only extract the columns in the tables that are needed.

Data minimisation is not applied on the following levels:

- Rows. We don't apply filtering on rows by default, unless there is a good reason (technical or functional). This is to avoid confusion about which data our data consumers are actually seeing.

Data minimisation use case

When you want to do a data minimization and specify one or more columns to the JSON file while loading the data, you can use the procedure `objectinsert`. Example for usage:

```
sql
WITH base AS (
  SELECT tryparsejson('{ "id":1,
                        "name": "ABC" }'
                    ) AS jsondata)
SELECT objectinsert(jsondata,'email','test@gitlab.com')
FROM base;

-- {
--   "email": "test@gitlab.com",
--   "id": 1,
--   "name": "ABC"
-- }
```

In the situation when you want to discard one or more columns from the JSON file while loading the data, you can use the procedure `objectdelete`. Example for using it:

```
sql
WITH base AS (
  SELECT tryparsejson('{ "id":1,
                        "name": "ABC",
```

```

        "address":{"add1":"1234",
                    "add2":"XYZ",
                    "city":"TN",
                    "apt": 1 }}'
        ) AS jsondata)
SELECT objectdelete(jsondata,'id','address')
FROM base;

-- {"name": "ABC"}

```

In this situation, you can exclude the column that shouldn't be processed for various reasons (RED data, PII data, no value for the data or other minimization principles).

Extraction solution

The Data Team has different instruments available to extract data out of source systems (randomly ordered):

- Custom made (via Python)
- Fivetran
- Meltano
- Snowflake data share
- Stitch
- Snowpipe
- Snowflake task

The decision for which instrument to use, is always based on the combination of:

1. Implementation effort.
2. Maintenance effort.
3. Ability to extend and adjust.

Its the Data Team that determines which instrument is used. The following decision diagram is used to determine the right extraction solution:

mermaid
graph LR

```

%%decisions
  apiavailable{API Available?}
  databucket{Data available in the gcp or s3 bucket?}
  fivetranconnectoravailable{Fivetran connector viable?}
  stitchconnectoravailable{Stitch connector viable?}
  dataisextsnowflakesources{Data is in different Snowflake Account}
  singeroption{Singer Tap viable?}
  streaming{Need to low-latency load?}
%%end solutions
  Custom([Custom development])

```

```

Fivetran([Fivetran])
Singer([Singer])
Stitch([Stitch])
Snowflakedatashare([Snowflake data share])
Snowpipe([Snowpipe])
Snowflaketask([Snowflake task])
%%flow
dsrequest[New Request]-->databucket
databucket-->|No|apiavailable
databucket-->|Yes|streaming
streaming-->|Yes|Snowpipe
streaming-->|No|Snowflaketask
apiavailable-->|No|dataisextsnowflakesources
apiavailable-->|Yes|fivetranconnectoravailable
stitchconnectoravailable-->|Yes|Stitch
fivetranconnectoravailable-->|Yes|Fivetran
fivetranconnectoravailable-->|No|stitchconnectoravailable
stitchconnectoravailable-->|No|singeroption
singeroption-->|Yes|Singer
singeroption-->|No|Custom
dataisextsnowflakesources-->|Yes|Snowflakedatashare
dataisextsnowflakesources-->|No|Custom

```

Custom development is a solution designed and developed by the GitLab Data Team. Examples of this are the current PGP and the Zuora Rev Pro extraction.

Periodically full refreshes (automatically)

There are several examples where data is not fully replicated in Snowflake. This occurs due to various reasons such as:

- Cursor fields not being updated.
- (Hard) deletes not being captured.

As a result, we need to perform full refreshes of certain tables periodically. Especially with SaaS tools like Fivetran and Stitch, there are limited possibilities to schedule regular full refreshes (e.g., once per month) for specific tables. The basic solution is manually triggering a full refresh, which varies in difficulty depending on the connector. When manual refreshes need to happen frequently or in an automated way, we should consider alternative approaches.

Alternative Loading Methods:

- Bypass Stitch or Fivetran's limitations by using Snowflake share (if it doesn't have the same limitations).
- Custom Pipeline: Build a custom data pipeline that allows for scheduled full refreshes.
- Upstream Governance: Explore data governance improvements in upstream systems to address the root causes of replication issues.

Each approach involves different trade-offs in terms of development effort, maintenance

requirements, and flexibility for our specific use cases.

Access request

Although it could be helpful to already provide the Data Team access to the source system, its not mandatory to raise an Access Request right now.

Work breakdown

End goal of the validation is to have a solution design and a complete work breakdown, with DRI's and size estimation attached. All the work that needs to be performed is stated down. We aim to have it on a level, where every work action could be converted to an issue with no larger T-Shirt size than M / 5/8 issue points.

MNPI Data

It needs to be determined if there is MNPI data in the data sources and if this data is about to be extracted towards the Data Warehouse. If the data source contains MNPI and this data is extracted, change the issue label of the issue to new data source MNPI.

workflow::3 - scheduling

Based on the business case, effort to implement, workload in the Data Team and Data Team priorities, the implementation will be scheduled. For scheduling we follow the GitLab Data Team Planning Drumbeat. This means that for every quarter, the Data Team determines when a new data source request will be picked up. When a new data source request remains in scheduling, it does not mean that it isn't on the radar of the Data Team. It means that it isn't been scheduled yet, because:

1. Defining the OKRs for the upcoming quarter didn't take place. Once a quarter the Data Team sets the OKRs for the next quarter.
2. It didn't fit in the OKRs for next quarter because of business case, effort to implement, workload in the Data Team and Data Team priorities.

workflow::4 - scheduled

When a request for implementing a new data source is in scope for next quarter OKRs, the request is scheduled. An epic will be created with the issue attached to it. Based on the work breakdown all issues are created with the corresponding issue weight, label assigned workflow::4 - scheduled, the right milestone (if known) and attached to the epic.

workflow::5 - development

When the execution of the work starts, the issue is in development and follows the regular development life cycle.

During the development, the Data Engineer aligns with all the stakeholders, about access to the data. Data access can be provided to the raw schema, depending on the data and the use case.

Creating labels

When it is a complete new data pipeline that extracts data from a upstream system that is not yet extracted, create a new label on in the GitLab Data group. I.e:

<https://gitlab.com/groups/gitlab-data/-/labels/23453371/edit>

workflow::6 - review

When the execution of the work is finished, the issue is in review and follows the regular development life cycle.

workflow::X - blocked

When the execution could not continue due to the need of external intervention, the issue is blocked. There must be a clear problem statement given and the right people need to be assigned on the shortest notice possible.

Red Data

Red data (according to the GitLab Data Classification Policy is not allowed to be stored in our Data Platform (Snowflake). Therefore we will not bring in/connect new data sources that are listed in the tech stack with dataclassification: Red unless there is a mission critical business reason. There is an exception process available which will enable us to evaluate the needs on a case-by-case basis and this process will require approval from BT/Data VP-level, Security and Privacy. Evaluating the business reason and obtaining approvals are part of the triage process and are governed via the new data source template.

Note: The exception process must be fulfilled to either connect a system with Red data and/or to extract Red data (fields) from that system. However, the business case to extract Red data (fields) under the exception process will necessitate a higher standard of review than a business case that only requires connecting a Red data system without extraction of Red data (fields). Where extraction of Red data (fields) is approved under the exception process, masking will be applied in the Data Platform (Snowflake) as described in the proceeding section.

Exception process - connecting a not listed source or a Red data source

When extracting new data towards Snowflake and the data source is not listed or listed dataclassification: Red;

- The team member must state a business case to connect the system to the Data Platform.
- The team member that requests the connection will perform an analysis of the data and confirm the data that is extracted is not Red data.
 - For instances where Red data (fields) must be ingested into Snowflake, masking will be applied upon extraction.
- BT/Data VP-level, Legal and Security must sign off to start the implementation.

Personal data

Extracting Personal Data into the Data Platform (Snowflake) is allowed, but it will require additional review from our Legal Privacy team and, where applicable, the People Team. When requesting to add a new data source, the team member that requests the upstream system/data

source is responsible to indicate if the data source contains personal data and which data elements it concerns. If a team member doesn't have enough knowledge to determine whether certain information is Personal Data, they should tag a subject matter expert and, if needed, a data engineer.

Please note that "pseudonymized" data is still Personal Data under privacy legislation.

If the upstream system (data source) contains Personal Data, the Data Team will either:

- exclude the indicated Personal Data elements from the extract if the extract can fulfill the requester's purpose without retaining the Personal Data; or
- obtain Legal Privacy review and approval to action the extraction to and processing of Personal Data in our Data Platform.

Sensitive Personal Data

Certain Personal Data elements are deemed "sensitive" under various privacy legislation. Sensitive Personal Data generally includes racial or ethnic origin, political opinions, religious or philosophical beliefs, trade union membership, genetic data, biometric data, data related to health, data related to sex life or sexual orientation, criminal offenses, and citizenship/immigration status.

Privacy legislation prohibits the processing of these types of data elements, except in limited circumstances. If a request is made to extract data that contains sensitive Personal Data, it will generally be denied unless true consent has been obtained to the processing, and Legal Privacy and, where applicable, the People Team, have approved this processing.

Monte Carlo observability

When a new data source is extracted towards the raw layer there will be a new separate schema created for that data source in many cases. In order to make sure the new data source is observed by Monte Carlo follow the steps outlined in Monte Carlo permission handbook section.

Documentation

The new data source will be documented in the handbook, following this template.

SECTION: enterprise-data/how-we-work/triage.md

*<i class="fas fa-users fa-fw color-orange font-awesome" aria-hidden="true"></i>*Data Triage

Enterprise Data Program Triage

GitLab has a robust and vibrant Data Program which includes a Central Data Team and many Functional Analytics Teams. GitLab total team members are growing as well and we need to uplevel our triaging process to keep up with GitLab's growth.

Steps to uplevel triaging process:

1. Incorporate Functional Analytics Teams into the data channel triaging process.
1. Create Slack Aliases so that GitLab Team members can ping a group of folks to get help with their questions in data.
1. Create a Triage Schedule for the Data Program to follow on the Data Program Calendar. Ask for a commitment of one team member, per Business Team, per day to help triage.

Triage Group Name	Triage Slack Alias	Triage Group Members
-----	-----	-----
Go To Market Analytics Triage	@GTMAAnalyticsTriage	Revenue Strategy & Analytics Team, Marketing Strategy and Performance Team, Business Insights and Analytics Team, GTM Data Fusion Team
R&D Analytics Triage	@R&DAnalyticsTriage	Product Data Insights Team, R&D Data Fusion Team
People Analytics Triage	@PeopleAnalyticsTriage	People Group Analytics Team, G&A Data Fusion Team
Engineering Analytics Triage	@EngineeringAnalyticsTriage	Engineering Analytics Team, G&A Data Fusion Team
Data Platform Triage	@DataPlatformTriage	Data Platform Team
Tableau Triage	@TableauTriage	Enterprise Analytics & Insights Team

- | Link to GitLab Data Projects |
- | ----- |
- | Data Team |
- | Sales Strategy and Analytics |
- | Marketing Strategy and Performance |
- | Product Data Insights |
- | Business Analytics |
- | People Analytics |
- | Customer Success Operations - Use CSAnalytics label |
- | Online Sales and Self-Service |

Slack Channel Alias Analytics Team Sunday Monday Tuesday Wednesday Thursday Friday
----- ----- ----- ----- -----
----- ----- -----
@GTMAalyticsTriage Revenue Strategy and Analytics @melia
Marketing Strategy and Performance @David Egan @rkohnke
Business Insights and Analytics @Vikas Agrawal
@R&DAnalyticsTriage Product Data Insights @Matthew Petersen @Carolyn Braza @Dave Peterson @Emma Neuberger @Nicole Hervas @Neil Raisinghani
@PeopleAnalyticsTriage People Analytics @Adrian Pérez
@DataPlatformTriage Data Platform Weekly rotation Weekly rotation Weekly rotation Weekly rotation Weekly rotation
@TableauTriage Analytics & Insights Weekly rotation Weekly rotation Weekly

rotation | Weekly rotation | Weekly rotation |
| @GTMAalyticsTriage @R&DAalyticsTriage @PeopleAnalyticsTriage | Analytics Engineering | |
Weekly rotation | Weekly rotation | Weekly rotation | Weekly rotation | Weekly rotation |

Data Platform weekly rotation schedule

In FY23-Q4, the data platform team moved to a weekly triage schedule. The schedule is maintained on the Data Program Google Calendar.

Tableau weekly rotation schedule

For days that show Weekly rotation on the Tableau triage schedule, i.e. Monday - Friday, the triage responsibilities will be rotated between Enterprise Data Analysts.

Analytics Engineering weekly rotation schedule

Beginning on 2025-01-02 Analytics Engineering will move to a weekly triage schedule. The schedule is maintained on the Data Program Google Calendar.

Enterprise Data Program Triage Instructions

All triage team members, regardless of their team, share the following responsibilities:

1. Triagers should review Slack messages in data
1. Triagers should respond in Slack threads by linking to relevant handbook pages, dashboards, or pointing team members to others who may have deeper expertise on the topic
1. If a request requires more than five minutes of investigation from a Data Program team member, triagers should direct the requestor to the channel description. The description includes links to the various Data Program projects where an issue can be created
1. Triagers should support team members who are new to the issue labeling process by reviewing issues with the clean-up::review label and offering guidance on proper labels to use
1. Triagers are responsible for monitoring and triaging all issues labeled with their team's label to ensure timely responses and resolution of team-specific requests.

A team member who is off, on vacation, or working on a high priority project is responsible for finding coverage and communicating to the team who is taking over their coverage; this should be updated on the Data Program's Google Calendar. To avoid putting the Monday workload on the same shoulders every week again, the Data Platform Team will rotate/exchange every now and then triage days in good collaboration on an ad-hoc basis.

Having dedicated triagers on the team helps address the bystander affect. The schedule shares clear daily ownership information but is not an on-call position. Through clear ownership, we create room for everyone else on the team to spend most of the day around deep work. The triager is encouraged to plan their day for the kind of work that can be accomplished successfully with this additional demand on time.

Triage responsibilities by team

Data triagers are the first responders to requests and problems for the Data Program.

Functional Analysts Responsibilities

- The Functional Analyst triage handle is primarily responsible for responding to GitLab team member requests that relate to their functions via Slack in data.

Data Platform Responsibilities

The Data Platform triager is responsible for resolving problems with our data platform, which will either be in the data-pipelines and data-prom-alerts slack channels, on the DE - Triage Errors board and on the MonteCarlo Incident page (make sure to filter on the Data Platform domain in MonteCarlo). Issues created from these errors should use the DE Triage Errors issue template.

- During the assigned triage week the Data Platform Team member will focused on (in priority order):
 - Incoming incidents
 - Open incidents
 - New issues: Every issue that comes in during a team member's triage week must be resolved.
 - A new issue is defined as an issue with no assignee and the workflow label triage. This issue list tracks these items.
 - Open issues on the Data Platform - Triage Errors board.
 - If an open incident or issue is already assigned it is still the triager responsibility to either take that issue or ensure progress is made.
 - If there is no work to be performed on incidents or issues on the board the triager will work on their regular work assignments.
 - Involvement from Data Platform Team members who do not carry triage responsibilities that week is likely still needed in some cases like:
 - A standing issue or incident could not be solved by the triager and triager need help from other Data Platform Team members.
 - Monitoring data-prom-alerts:
 - The data-prom-alerts slack channel is used for the most urgent breaking events, which requires immediate action. It is the responsibility of all Data Platform Team members to ensure action is taken in time after office hours of the triager.
 - Assistance from the Data Platform Team is needed by other GitLab Team members and this is outside of the office hours of the triager.
 - Monte Carlo incidents are posted in the data-pipelines Slack channel (except schema changes). Because Monte Carlo only initially notifies an incident, checking the Monte Carlo Incident page is needed to avoid missing any incidents. Schema changes are filtered out using this link, because these don't require action (and are also not reported in the Slack channel). All Monte Carlo incidents need to be given an appropriate resolution status or linked to an assigned GitLab issue end of day.
 - Note: Currently there is a large backlog of unclassified incidents in MonteCarlo. We currently focus only on the last 7 days.

Analytics Engineering Responsibilities

The Analytics Engineer triager has two primary responsibilities: addressing dbt-related errors (both new and existing) and supporting issue organization.

They monitor the analytics-pipelines Slack channel for dbt-test, dbt-run errors and MonteCarlo

anomalies, creating new issues using the AE Triage Errors issue template when needed.

BI / Tableau Team Responsibilities

The BI / Tableau triager is primarily responsible for acting as the primary owner of responses to ad hoc questions in the data-tableau channel (others will continue to pitch in, but the person on triage will be the DRI) and facilitating Tableau Office Hours that week

Data Science Responsibilities

The Data Science triager is primarily responsible for reviewing model run issues / breakages and supporting operational requests such as executing field replacements due to dbt model updates.

We will iterate on triage responsibilities to include additional activities such as extract refresh failure review, job failure review, etc. as the team expands.

- For more information on responsibilities of a triager watch the Data Engineer triage training session video.

How to Complete Data Triage

For issue triaging, the triager should add the appropriate labels to the issue to put the issue into the respective project's workflow. It is not the responsibility of the triager to validate the issue and determine any root causes or solutions. Rather, for the Data Team project for example, the triager should add the appropriate scoped TEAM label (Ex.Team:GTM) and the scoped Workflow::triage label. From there, those issues are then put into the respective team's workflow.

Note:

The Triager:

- is not expected to know the answer to all the questions
- should cc (mention) other team members who are subject matter experts (SME) or functional DRIs for their feedback
- should document any issues stumbled upon and help to disseminate knowledge amongst all team members

The Central Data Team triager will create an issue in the Data Team project. Task and duties are stated in the issue template.

Read the FAQ and common issues.

New issue triaging for the Data Platform Team

Every issue that comes in during a Data Platform Team member's triage week must be resolved as follows:

1. All issues are assigned to triager as starting point.
1. All issues are triaged, validated and refined - following the defined workflow (criteria).

If the triager cannot prepare it adequately:

- Assign to a team member with domain expertise, OR
- Assign to Director Data Platform if appropriate expertise is unknown

1. If an issue is 1-2 issue points they will fully implement the solution. This means moving with through all workflow stages up until workflow::6 - review)

1. If an issue is 3 or more issue points, the issue will be labelled as workflow::4 - ready to develop.

- If an issue is not urgent. Triager unassigns themselves and issue has been placed in the backlog.

- If an issue is urgent. Triage alligns with a team member on assignment or assigns to Director Data Platform.

mermaid

flowchart TD

```
A[New Issue] --> B[Assign issue to themself]
B --> C[workflow::1 - triage]
C --> D[workflow::2 - validation]
D --> E[workflow::3 - refinement]
F[Assign to other Team Member or Data Platform Team Lead ]
C -->|If unable to perform| F
D -->|If unable to perform| F
E -->|If unable to perform| F
E -->G{Can complete}
G -->|No and urgent| F
G -->|No not urgent| H[workflow::4 - ready to develop]
G -->|Yes| J[workflow::5 - development]
J -->K[workflow::6 - review]
U[Unassign]
F-->U
H-->U
K-->U
```

Incident

The Data Team follows the incident definition from Engineering: Incidents are anomalous conditions that result in·or may lead to·service degradation or outages. These events require human intervention to avert disruptions or restore service to operational status.

Service degradation or outages in data can be seen as:

- Data is not available
- Data is not correct
- Data is outdated
- Data leakage

This means the following events (not extensive) are likely to be incidents:

- DBT model failed (and downstream models are skipped)
- DBT test failure

- DBT source freshness failure
- Infrastructural incident (Snowflake not accessible, Tableau not accessible)
- Data source not available
- Data pipeline not running

Severity

Depending on the nature and impact of the incident a severity needs to be determined. Currently we don't have a decision matrix in place. To determine the severity, take the following aspects into consideration:

- In case of a data source or data pipeline incident, check the Data Source Tier.
- Number of users impacted
- Impacting Trusted Data models

Follow up

- Incidents are always given immediate attention.
- Every incident has a DRI assigned. This is not necessarily the triager/creator of the incident. Due to the nature of asynchronous working at GitLab, the triager/creator is the DRI until another GitLab Team Member is actively contacted/involved.
 - The codeownerfile is the right futuresource to find the right DRI for assigning the incident. Currently the code ownership is not well defined. As part of FY23-Q1 we are planning to have a more strict ownership.
- Every raised incident will be communicated in the data Slack channel, followed by a short description, ETA and link to the incident. The right GitLab Team Members are tagged.
 - A regular (depending on the severity) update is posted in Slack. Sometimes there isn't a new status, don't hesitate to communicate this as well.
 - Timelines should be documented in the timeline section under the incident Subject/Header for use in retrospectives and other investigations.
 - When the incident is solved, an update is posted in Slack

Triage Bot

The Data Team uses the GitLab Triage gem to automate issue management and keep the analytics project organized. The triage policy for the analytics repo is defined in the .triage-policies.yml file.

Label Enforcement

Triagers use labels to identify and prioritize work within their domain. If you're unsure which labels to apply to an issue, just add the clean-up::review label and the team will help with proper labeling. This automation ensures that every issue includes the required scoped labels: team, work category, champion, and workflow. Labels like Documentation, Iteration Planning, and Discussion are excluded from this requirement.

- After 3 days: If an issue is missing required labels, the bot adds a comment listing the missing labels and applies both Needs Triage and clean-up::warning. The comment includes instructions on how to ask for help using the clean-up::review label.
- After 14 days: If the labels are still missing, the bot adds a reminder comment.

- After 30 days: If there's still no update, the issue is automatically closed. The bot adds the `clean-up::close` label and a comment explaining why. Team members can reopen these issues at any time. To prevent future auto-closure, make sure to add the required labels.

When a closed issue is reopened, the bot removes the `clean-up::close` label and checks for the required labels. If any are still missing, it adds `clean-up::warning` and includes a comment listing what's needed.

Once the correct labels are in place, the bot automatically removes any warning labels. This creates a self-maintaining system where issues either get labeled correctly or are closed for review.

Stale Issue Management

The triage bot flags issues that haven't had any activity in over a year to help keep the backlog manageable. When that happens, it adds the `stale::warning` label and posts a comment. From that point, the issue has 14 days before it is automatically closed unless someone takes action.

To prevent an issue from being closed, you can do one of the following:

- Leave a comment on the issue with an update on its current status, then remove the `stale::warning` label.
- Add the `stale::exempt` label if the issue should remain open without needing regular updates

Just adding a comment won't stop the process. The warning label needs to be removed or replaced with `stale::exempt`.

Here's how the timeline works:

1. After 1 year of inactivity: The bot adds `stale::warning` and posts a comment
2. 7 days later: It adds `stale::7daywarning` as a final reminder
3. After another 7 days (14 since initial warning): The issue is closed and tagged with `stale::closed`

Closed issues can be reopened at any time. Once reopened, make sure to update the issue or apply the `stale::exempt` label so it is not flagged again in the future.

Testing Policy Updates

To test changes to the triage policy file, run the `dry-run:triage` CI job in the Stage: Triage of your merge request. This job will not make any actual changes but simulates the outcome of applying the policy file and prints what actions would be taken.

> Important: A successful job (green check) only means the bot ran without errors. It does not mean your triage rules are correct or effective. You must open the job logs and carefully review the output to confirm that the rules match your expectations.

The logs will show:

- Which triage rules were triggered
- How many issues matched each rule
- What actions would be taken (like labels added, comments posted)

End of day wrap-up

In order to get better and be more efficient in daily triage, we wrap-up the work by the end of the day. The following information is provided by the Data Analyst each day:

- Time Tracking: Listing down the time spent that day on triage and which activities are performed. The general idea is to gain an understanding on where the workload is, in order to optimize those activities. Please list down the work so it is useful for analysis purposes.

I.e:

- 2 hours solving data ingestion issue, for source x
- 1 hours solving data transformation issue
- 1 hour on solving a broken Tableau Dashboard
- 1 hour triaging and re-routing (new) issues
- 0.5 hour on answering Slack questions
- Groundhog Issues. Issues that occur on a regular basis are annoying and costing us unnecessary time. List down any issue that is popping up time after time. This is in order to find any spots in the current landscape to address.

A triage roundup will take place on a weekly or twice-weekly basis in order for the data leadership team to consolidate triage efforts. Please bear in mind the purpose of the information provided, to make it useful and improve Triage.

End of week wrap-up

The Data Platform Team follows a weekly rotation schedule which means that by the end of the triage week the triager will hand over the triage responsibilities.

- Even though we still use the triage issue on a daily basis, the triager will only write up the week a-sync in the central data team triage issue on Friday end of day.
- The triager will report/verbalize in the Weekly Data Platform Team meeting any notable things happened on triage on Tuesday in the next week.

Although running a weekly rotation, we expect the triager to post an EOD announcement in the applicable Slack channels.

GitLab.com databases structure changes

GitLab.com databases do regularly change. In order not to break the daily operation, changes to the database needs to be tracked and checked. Any change to the GitLab.com database and CustomerDot database is tracked by the Danger Bot. The Data Team gets notified, by applying labels to the MR, if a change to the db/structure\sql is made.

A label Data Warehouse::Impact Check is added by the Danger Bot as call to action for the data team.

- On triage, the Triager will check for MRs with label Data Warehouse::Impact Check.

The following actions are performed by Data Team Triager:

- Every merge request (MR) will be judged
 - If MR contains the label group::product intelligence along with Data Warehouse::Impact Check, there are a couple of checks that need to do:
 - Because a new metric is added or the existing one is altered, the Data team should ensure the change will not break the Service ping extraction process
 - Check new metric SQL statement from the original MR (a typical example is gitlab-org/gitlab/mergerequests/75504) and execute it on Snowflake - usually, it is just a SELECT SQL statement
 - If the changes to the SQL file are not causing a break in the operation, the label will be changed to Data Warehouse::Not Impacted.
 - If the changes to the SQL file causing a break in the operation:
 - The Label will be changed to Data Warehouse::Impacted
 - A new issue is opened in the GitLab Data Team project, assigned to the correct DRI and linked to the original MR.
 - Impact will be determined in the issue.
 - Any MRs will be created to overcome loading issues, downstream dbt processing and Tableau usage.
 - If impact is beyond data loading, this means the data is used downstream, an Analytics Engineer must be included to also determine the business impact of the upstream change.
 - According to the Merge of the GitLab.com MR, merge will be planned.
 - If the MR does not contains the label group::product intelligence and it concerns changes to SQL structure:
 - Check if it will break the operation / data pipeline, following the Determination matrix below.
- If any MR will cause a break in the operation, the label will be changed to Data Warehouse::Not Impacted.
- If any MR will cause a break in the operation:
 - The Label will be changed to Data Warehouse::Impacted
 - A new issue is opened in the GitLab Data Team project, assigned to the correct DRI and linked to the original MR.
 - Impact will be determined in the issue.
 - Any MRs will be created to overcome loading issues, downstream dbt processing and Tableau usage.
 - If impact is beyond data loading, this means the data is used downstream, an Analytics Engineer must be included to also determine the business impact of the upstream change.
 - According to the Merge of the GitLab.com MR, merge will be planned.
 - All stakeholders will be informed.

Graphical representation of the process

<details><summary>Click to expand graphical representation of the process</summary>

mermaid

flowchart TD

subgraph "Original MR"

CHECKBOARD

ADDITIONALLABEL

```

CLABEL
CHANGELABELI
CHECKDDL
end
CHECKBOARD(Check MRs on the board) --> ADDITIONALLABEL{Does MR has a label: group::product
intelligence}
ADDITIONALLABEL --Yes--> CHECKORIGINALISSUEPI{{Check code in the original MR}}
ADDITIONALLABEL --No--> CHECKDDL{Will DDL break in the operation}
CHECKDDL --Yes--> CHANGELABELI(Change label to Data Warehouse::Impacted)
CHECKDDL --No-->CLABEL(Changed label to Data Warehouse::Not Impacted)
CHANGELABELI-->OI(Open an new issue is opened in the GitLab Data Team project)
subgraph "Data team project"
  OI-->IM(Impact will be determined in the issue)
  IM-->CHECK(Check downstream dbt processing and Tableau usage)
  CHECK-->PL(Plan MR)
  PL-->INFORM(Inform stakeholders)
end
INFORM-->END((End))
CLABEL-->END
OKSQL--Yes-->CLABEL
OKSQL--No-->CHANGELABELI
subgraph "group::product intelligence"
  CHECKORIGINALISSUEPI -->SQL[(Find and execute SQL statement)]
  SQL--Execute-->OKSQL{Is SQL executed properly}

end
end

```

</details>

Determination matrix:

```

| Change | Call to action needed |
| ----- | ----- |
| New table created | :x: |
| Table deleted | :whitecheckmark: |
| Table renamed | :whitecheckmark: |
| Field added | :x: |
| Field removed | :whitecheckmark: |
| Field name altered | :whitecheckmark: |
| Field datatype altered | :question:|
| Constraints changed | :question: |

```

We are not loading all the tables and columns by default. Thus if new tables or columns are added, we only will load these tables if there is a specific business request. Any change to the current structure that causes a potential break of operation needs to be determined.

Determination matrix is not extensive. Every MR should be checked carefully.

Triage common issues

In this section we state down common issues and resolutions

GitLab Postgres Database not accessible

In a situation when gitlabdotcom postgres replica snapshot is not built correctly, the task checkreplicasnapshot inside the extract DAG's fail for MAIN and/or CI db, which indicates either that the replica snapshot is not rebuilt/accessible or the pglastxactreplaytimestamp value was not present. The error message generated from task failure would indicate errors with the replica database connectivity or database system starting up and the error in the task would look something similar like this:

console

```
[2023-07-15 13:04:48,022] INFO - b'psycpg2.OperationalError: could not connect to server:
Connection refused\n'
[2023-07-15 13:04:48,022] INFO - b'tls the server running on host "{dbinstanceip}" and
accepting\n'
[2023-07-15 13:04:48,022] INFO - b'tTCP/IP connections on port {port}?
```

Follow the runbook for the steps to perform, including communication.

Automated service ping issue

In a situation when Service ping fail while it generates metrics, we should be informed either via Trusted data dashboard or Airflow log - generally, the error log is stored in RAW.SAASUSAGEPING.INSTANCESQLERRORS table. Follow the instructions from the link [error-handling-for-sql-based-service-ping](#) in order to fix the issue.

Zuora Stitch Integration single or set of table-level reset

It could happen, in any case, to reset the table in Stitch for the Zuora data pipeline, in order to backfill a table completely (i.e. new columns added to in the source, technical error etc).

Currently, Zuora Stitch integration does not provide table level reset, and thus we have to perform a reset of all the tables in the integration. This will result in extra costs and risks.

To this below steps can be followed using which we have successfully done the table level reset.

In this example, we have used Zuora subscription table, but this could be applied to any other table in the Stitch Zuora data pipeline.

Step 1:- Rename existing table with the date suffix to identity the backup, recommended format YYYYMMDD

sql

```
ALTER TABLE "RAW"."ZUORASTITCH"."SUBSCRIPTION" RENAME TO
"RAW"."ZUORASTITCH"."SUBSCRIPTION20210903";
```

Step 2:- Pause the regular integration

Step 3:- Create a new integration Zuora-Subscription in Stitch

While setting it up setup the extraction frequency to 30 minutes and date from extraction to 1st Jan 2012 to ensure all data gets pulled through.

Step 4:- Run the newly created integration

Try running the newly created integration manually and wait for it to complete. Once completed then and it shows on the home page successfully. Once done Pause the newly integration task because we don't want any misaligned data while we follow the next steps.

Step 5:- Check for the records

In the newly created table "RAW"."ZUORASUBSCRIPTION"."SUBSCRIPTION" cross-check the number of rows showing as loaded in the integration UI in stitch and loaded in the table is same.

Step 6:- Create the table in the main schema

Move the newly loaded data to ZUORASTITCH schema because the new integration will create the table in the ZUORASUBSCRIPTION as stated above in the image.

sql

```
CREATE TABLE "RAW"."ZUORASTITCH"."SUBSCRIPTION" CLONE  
"RAW"."ZUORASUBSCRIPTION"."SUBSCRIPTION";
```

Note: Check for the primary key present in the table post clone or not if not check for the primary key in the link and add the constraints on those columns.

Step 7:- Make records count check to ensure we don't have fewer records in the new table

sql

```
select count() from "RAW"."ZUORASTITCH"."SUBSCRIPTION20210903" where deleted = 'FALSE';  
select count() from "RAW"."ZUORASTITCH"."SUBSCRIPTION" ;
```

Step 8:- Drop the new schema

sql

```
DROP SCHEMA "RAW"."ZUORASUBSCRIPTION" CASCADE ;
```

Step 9:- Delete temp Zuora-Subscription integration and enable regular integration

Step 10:- Run regular integration and validate

This is to ensure that error observed previously to the table is gone and data is getting populated in the table.

Check on duplicate ids due to 2 different extractors, to ensure the data is getting populated in the table correctly.

sql

```
select id, count() from "RAW"."ZUORASTITCH"."SUBSCRIPTION"  
group by id  
having count() > 1
```

Note Refer to the MR (internal link) for more information.

Source freshness errors

See the source contact spreadsheet for who to contact if there are external errors related to a source.

Airflow Task failure

| |

| ----- |

| DAG gitlabcomdbextract
 Task gitlab-com-dbt-incremental-source-freshness
 |

| Background: This extract relies on a copy (replication) database of the GitLab.com environment. Its high likely that this is the root cause of a high replication lag%20and%20on(i nstance)%20(pgreplicationisreplica%7Btype%3D~%22postgres-

(archive)%22%7D%20%3D%3D%201)&g0.tab=0&g0.stacked=0&g0.rangeinput=1w&g1.expr=pglongrunning actionsageinseconds%7Btype%3D~%22postgres-

(archive)%22%7D&g1.tab=0&g1.stacked=0&g1.rangeinput=6h). |

| More information of the setup here (internal link). |

| Possible steps, resolution and actions: - Check for replication lag
 - Pause the DAG if needed
 - Check for data gaps
 - Perform backfilling
 - Reschedule the DAG |

| Note: The GitLab.com data source is a very important data source and commonly used. Please inform an update business stakeholders accordingly. |

Sheetload - Column 'REF!' is not recognised

| |

| ----- |

| DAG sheetload
 Task dbt-sheetload
 |

| Background: This is an issue with Google sheets when data is being imported from a second sheet using Google sheets' import function. Occasionally the connections between the sheets stop working and the sheet needs to be refreshed. |

| More information of the setup here. |

| Possible steps, resolution and actions:
 - In general you should just need to open the Google sheet which is failing and confirm the data has been re-populated.
 - If you do not have access to the sheet contact @gitlab-data/engineers and confirm if anyone else does. |

Model versionusagedataunpacked stale

When got an error for model versionusagedataunpacked and error looks like:

console

```
[2022-01-26 11:56:32,233] INFO - b'\x1b[33mDatabase Error in model versionusagedataunpacked
(models/legacy/version/xf/versionusagedataunpacked.sql)\x1b[0m\n'
[2022-01-26 11:56:32,233] INFO - b' 000904 (42000): SQL compilation error: error line 241 at
position 12\n'
[2022-01-26 11:56:32,233] INFO - b'" invalid identifier '{metricsname}'\n"
[2022-01-26 11:56:32,233] INFO - b' compiled SQL at
target/compiled/gitlabsnowflake/models/legacy/version/xf/versionusagedataunpacked.sql\n'
[2022-01-26 11:56:32,234] INFO - b'\n'
```

The root cause of this issue is when new metrics are introduced in an upstream model - and this model (along with model versionusagedataunpackedintermediate) try to pivot values to columns. Without full refresh, this will not happen under the pipeline.

Full refresh required as per instructions from dbt models full refresh - internal handbook.

An example for this failure is the issue: 11524 (internal link)

Zuora Revenue Source and Target Column Mismatch

Sometimes Zuora Revenue source system as part of a certain release modify the source table definition by adding/removing column(In a year once or twice). Whenever this type of change happens it triggers failure in the loading task in DAG zuorarevenueloadsnow.

For example, there was 3 additional column added to table BI3RCPOB which lead to the below error message.

console

```
[2022-03-21, 13:26:48 UTC] INFO - sqlalchemy.exc.ProgrammingError:
(snowflake.connector.errors.ProgrammingError) 100080 (22000):
01a31326-0403-d02c-0000-289d37f10d4e: Number of columns in file (102) does not match that of
the corresponding table (99), use file format option erroroncolumncountmismatch=false to ignore
this error
[2022-03-21, 13:26:48 UTC] INFO - File 'RAWDB/staging/BI3RCPOB/BI3RCPOB12.csv', line 3,
character 1
[2022-03-21, 13:26:48 UTC] INFO - Row 1 starts at line 2, column "BI3RCPOB"[102]
[2022-03-21, 13:26:48 UTC] INFO - If you would like to continue loading when an error is
encountered, use other values such as 'SKIPFILE' or 'CONTINUE' for the ONERROR option. For more
information on loading options, please run 'info loadingdata' in a SQL client.
```

When this kind of failure happens we need to alter/create or replace the target table and add the additional column to the table definition in RAW.ZUORAREVENUE schema. It is good to practise creating or replacing because it requires doing the full refresh of the data because of the addition of a new column.

Below are set of steps that will guide you to resolve this.

Step 1:- Download the first file from the storage to local to view the additional column. For

any file, only the file name and folder name should be modified.

console

```
gsutil cp gs://zuorarevprogitlab/RAWDB/staging/BI3RCPOB/BI3RCPOB1.csv .
```

For any other file, only the file name and folder name should be modified. For example table, BI3RCBILL above command will change to gsutil cp
gs://zuorarevprogitlab/RAWDB/staging/BI3RCBILL/BI3RCBILL1.csv .

Step 2:- Once the file is downloaded look for the header of the file using the below command

console

```
head -1 BI3RCPOB1.csv
```

Step 3:- Pull out the current table definition from snowflake using the below query

sql

```
USE ROLE LOADER;  
USE DATABASE RAW;  
USE SCHEMA ZUORAREVENUE;  
select getddl('table','BI3RCPOB');
```

Prepare the CREATE OR REPLACE TABLE query by comparing the missing column from the header and the table definition and adding them to the new table. The missing column should be at the end and the data type should be varchar.

Deploy the modified SQL.

Note:- If the table doesn't exist then create the table with all the column named present in header with datatype as Varchar.

Step 4:- Move the log file from the process folder to the staging folder of the table.

console

```
gsutil cp gs://zuorarevprogitlab/RAWDB/processed/22-03-2022/BI3RCPOB/BI3RCPOB22-03-2022.log  
gs://zuorarevprogitlab/RAWDB/staging/BI3RCPOB/
```

The point to consider in this command is the date in the path and the log file name. If the failure happened on 23-03-2022 then it will become

```
gs://zuorarevprogitlab/RAWDB/processed/23-03-2022/BI3RCPOB/BI3RCPOB23-03-2022.log
```

Validate in GCS storage)&prefix=&forceOnObjectsSortingFiltering=false) that the log file is present for the respective table.

Step 5:- Re-run the task from the airflow by clearing the task.

Triage FAQ

Is Data Triage 24/7 support or shift where we need to support it for 24 hours?

We need to work in our normal working hour perform the list of task mentioned for the triage day in the Triage Template (internal link)

If any issue is found do we directly jump to fix it in production or take it as part of the incident and solve it within the defined time?

On the Triage day the data team member present will look for all the failures, questions or errors in:

- The Slack-channels; data-prom-alerts analytics-pipelines and data
- Newly added issues

It includes all the failures since the last person did sign off and will create an issue for all the failures since then till the person signs off.

If any data pipeline has broken and there is expected to be a delay in getting data loaded or refreshed. The concerned team has to be notified using the Triage Template (internal link)

Is there ETA for a different kind of issue?

If the pipeline is broken it needs to be fixed, currently we are working on defining SLO's for our data assets. For our data extraction pipelines, there is a comprehensive overview here.

If I work my normal hours on triage day i.e. till 11 AM of US timeline. What happens when the pipeline breaks post my normal hours and there is a delay in data availability?

Yes, the benefit of our presence is that we have a wide overage of hours. If the person who is on Triage is ahead of US timelines, we have an advantage of solving issues timely. The downside is that we have not full coverage that day for US timelines. This is an attention point towards the future.

Useful regex

Match lines where these terms do not exist

`^(?!(<First term to find>|<Second term to find>)).$`

e.g. For cleaning up Airflow logs:

`^(?!.(Failure in test|Database error)).$`

SECTION: enterprise-data/how-we-work/data-development/index.md

Data Development at GitLab

GitLab deploys two distinct but interrelated approaches to build data solutions that help drive insights and business decisions. These approaches are complementary to one another and are focused on delivering results at a level of speed, quality, and reliability required by the business, problem being solved, and question being asked. The approaches are complementary and evolutionary in nature, enabling development in an earlier stage to be leveraged in a later stage if required. Data solutions developed at an early stage can be improved and enhanced to a

later stage if there is sufficient business need to do so. All analysis follows the well-established Data Analysis Process.

Development Location per Data Application

	Ad-hoc Development Location	Trusted Data Development Location
--	:-:	:-:
Snowflake	WORKSPACE Schema	COMMON and SPECIFIC Schemas
Tableau	Development Project Folders	Production Project Folders

Development Location Comparison

	Ad-hoc	Trusted Data
--	:-:	:-:
When To Use	Prototyping / Directional / Urgent Analysis	Mission Critical Analysis / Operational Analysis
Snowflake Schema Location	WORKSPACE	COMMON and SPECIFIC
Tableau Folder Locations	Development	Production
Manual adding data	optional	N/A
Creating own data structures	optional	N/A
Visualization using Tableau	optional	required
Built Using the Enterprise Dimensional Model	optional	optional
Built Using Data from the COMMON or SPECIFIC schemas	optional	required
Registered in the Data Catalog	N/A	required
Follows Trusted Data Development process	N/A	required
Tested using the Trusted Data Framework	N/A	required
Auditable w/linkage to source systems	N/A	required
Trusted Data Branded	N/A	required

Data Development

Ad-hoc Data Development

Ad-hoc delivers a report or dashboard for one-time or limited use and it can also deliver a prototype, first iteration of a data solution that is not mature enough for a long-term, Trusted Data Solution yet. Ad-hoc development is performed when no existing data solution answers the questions being asked. Code developed for ad-hoc analysis for one-time or limited use is not written to be leveraged in a long-term solution; rather, it is a means to deliver results quickly. Code developed for ad-hoc analysis for prototypes and first iterations of data solutions can be leveraged in a long-term, Trusted Data Solution.

To complete ad-hoc analysis, Analysts typically write and run SQL queries against the Enterprise Data Warehouse and extract data to analyze using tools like Tableau or Python. Analysts and Analytics Engineers can also complete Ad-Hoc analysis using dbt and prototyping data solutions that can be leveraged in long-term, Trusted Data Solutions. At times, new data may need to be sourced from text files, spreadsheets, or other data sources. In many cases, the ad-hoc report solves for the immediate business need and no further action is required. However, sometimes the results of ad-hoc analysis yield results that require additional data modeling or dashboard development. In these cases, a more robust and trustworthy solution can be developed using Trusted Data Development.

Ad-hoc development gives ultimate flexibility to prototype data solutions. If a new data set needs to be explored and new transformations need to be built in a fast-paced, iterative way, the Ad-hoc Data Development approach can be used. Because of its flexible nature, not all ad-hoc data development is suitable to make mission critical decisions. It is often a first step for maturing the data solution. Care and caution should be taken when using ad-hoc data solutions to inform business making decisions.

Ad-hoc development takes place in the WORKSPACE or EXPLORATIONAL schemas where the following access and permissions apply:

- WORKSPACE Schema: Users lack create/insert/update/drop permissions. Updates are made via standardized dbt processes.
- EXPLORATIONAL Schema: Users have create/insert/update/drop permissions, allowing for rapid prototyping. This schema is less governed and primarily used by a smaller, trusted audience.

Trusted Data Development

Trusted Data delivers the most complete, reliable, and accurate analytics available to an enterprise. Over time as an organization matures and value of analytics increases, Trusted Data evolves and development rigor also evolves, but the core steps remain consistent and include requirements gathering, design, iterative wireframing, testing, and operational monitoring. Trusted data solutions differ from ad-hoc reports because they include quality validations such as data testing, code review, and registration in the Data Catalog. Trusted data solutions can be built either in the Enterprise Dimensional Model (EDM) located in the COMMON schemas or in the SPECIFIC schema that models application data. Both solutions are trusted due to having data testing, code review, and registration in the Data Catalog.

Trusted data development uses the following schemas:

- COMMON Schema: Models cross-functional business processes using Kimball's dimensional modeling techniques which allows joining of data in a star schema structure for robust and easy to data modeling.
- SPECIFIC Schema: Represents not cross-application or cross-functional data. Overtime, these data models may mature and evolve into cross-functional models. (e.g. NetSuite data source).

Trusted Data Development Steps

Trusted Data Solution Criteria

All Trusted Data solutions must meet the following criteria:

1. The business problem is defined with clear revenue impact established
1. A Data project Epic is created to manage development
1. Requirements and success criteria are captured and tracked in the Epic
1. Scope is defined for v1.0 and v1.1, with a release cycle established up-front (e.g. weekly, bi-weekly, monthly)
1. A Dashboard Wireframe is created in Lucid or Tableau and shared with users, iterating to "final draft"

1. If a Dimensional Data Model solution is indicated, then the Dimensional Data Model is designed and integrated into the Enterprise Dimensional Model Bus Matrix. This step is not applicable for data modeled in the SPECIFIC application schema.
1. (DRAFT: Under review with Monte Carlo Project) Trusted Data Tests are created and deployed
1. The solution enters a User Acceptance Testing phase, including data validations to source systems
1. The solution is registered in the Data Catalog
1. The solution is deployed, including any required training and user enablement

Trusted data development prioritizes consistency, reliability, and adherence to industry best practices.

Database Implementation

In both Ad-hoc and Trusted Data development data is made available in the PROD database, via multiple schemas. Data can be read out of multiple schemas in Snowflake and Tableau.

Ad-Hoc Database Implementation

To make data available for Ad-Hoc Data Development, data is transformed and made available in the Snowflake PROD database. This data is available in two different schemas, WORKSPACE AND EXPLORATIONAL.:

- WORKSPACE Schemas: Prefixed with WORKSPACE or RESTRICTEDSAFEWORKSPACE variants for MNPI data. These schemas require standardized dbt processes.
- EXPLORATIONAL Schemas: Prefixed with EXPLORATIONAL or RESTRICTEDSAFEEXPLORATIONAL variants for MNPI data. These schemas require read and write permissions in order to create tables, add columns and prototype data solutions. The schemas are set up on departmental level and access is not provisioned on a lower grain than schema level. Functional ownership of the schemas resides with the departmental VP (or equivalent). This means that the VP needs to provide approval in case of an Access Request and carries the responsibility of proper usage of the data in the schema (i.e. in case of MNPI, PII and sensitive data). EXPLORATIONAL schemas are the least governed data schemas, which gives ultimate flexibility. Advise to use these schemas on a need-to-use basis.

Trusted Data Database Implementation

Trusted data resides exclusively in Snowflake's PROD database. It follows the EDM methodology or the Specific application methodology and adheres to stringent development standards:

- Schemas: Prefixed with COMMON or SPECIFIC, with RESTRICTEDSAFE variants for MNPI data.
- Methodology: Data transformations use dbt to align with either the Enterprise Dimensional Model(fact and dimension tables) or Specific application tables.
- Governance: Trusted data undergoes rigorous validation, documentation, and registration.

BI Tool Implementation - Tableau

Tableau leverages the robust data warehouse structure, configuration, and security to deliver high-quality, reliable analytics. This integration ensures optimized data retrieval, governance, and scalability, empowering organizations to create accurate, insightful dashboards while maintaining data integrity. Tableau classifies data with Project Folder environments and

Certified Data Source Data Labels.

Development and Production Project Folders

- Development Project Folders: Are a prototyping and testing environment which Supports iterative work with evolving datasets and dashboards. This Sandbox environment allows creativity and flexibility during design phases. Further documentation can be found [here](#).
- Production Project Folders: Hosts trusted data and validated , finalized dashboards. This ensures end-users access validated, high-quality content which maintains data integrity and supports decision-making. Production Folder Publishing requirements can be found [here](#).

Tableau Certified Data Sources

Certified data sources serve as reliable, reusable, and versatile single sources of truth for Tableau reporting. These data sources are reviewed for accuracy, compliance, and broad usability. By adhering to best-in-class BI and modeling standards, certified data sources streamline onboarding, reduce duplication, and promote consistency, simplifying decision-making and reinforcing their role in the organization's data ecosystem. Learn more about Certified Data Sources by [clicking here](#).

SECTION: enterprise-data/how-we-work/functional-analytics-center-of-excellence/_index.md

Welcome to the FACE of Data Handbook

{{% alert color="success" %}}

The FACE (functional analytics center of excellence) is a cross-functional group of functional analytics teams that aim to make our teams more efficient by solving and validating shared data questions which results in cohesive measurement approaches across teams.

{{% /alert %}}

Context

The central data team serves as the hub for all of our "spoke" functional analytical teams; however, we have an opportunity to establish spokes between the spokes. Enter: the creation of the functional analytics center of excellence (FACE).

FACE Teams: what functional ana